

RMA-Dokumentation

1 Konzept

Genutzte Technologien:

- HTML, CSS, JS
- Phaser.js
 - Laden der Bilddateien
 - Statt dem Canvas Image-API (wie im Skript vorgestellt) wird die **„this.load.image()“** – Funktion von Phaser genutzt
 - Platzieren der Spielobjekte
 - Um die Spielobjekte je nach Level zu platzieren, sollen die Positionen und die Objekt-Dateien in einer JSON-Datei definiert werden, die anschließend in Phaser geladen wird.
 - Steuerung
 - **Physics-Komponente** wird genutzt, um Kollisionen, Bewegungen und Physikregeln für den Spieler/die NPCs zu definieren
 - Animation
 - **„load.spritesheet()“**: laden von Sprite Sheets, die aus einer Abfolge von Frames bestehen
 - **anims-API** zur Umsetzung von Animationen
 - Hierbei wird eine Animation mit `this.anim.create` definiert und anschließend von Phaser während der Rendering-Loop abgespielt. Somit muss kein Rendering-Loop erstellt werden, wie es im Skript gezeigt wurde, da Phaser dies übernimmt
 - Sound (in Soll-Anforderungen)
 - Implementierung erfolgt über das **Audio-Modul** von Phaser statt dem „<audio>“-Element wie es im Skript vorgestellt wurde

Muss-Anforderungen:

- Start-Szene
 - Start-Button
 - Info-Button (Stellt Hinweis zum Ziel des Spiels, Story und Hindernissen)
 - Story:
 - PC-Spieler verpasst Schulbus und darf nicht zu spät zur Schule kommen. Während des Laufes muss der Spieler über Hindernisse springen bzw. ihnen Ausweichen
 - Hindernisse (NPC):
 - Passanten (bewegen sich nach links & rechts): Bei Kollision verliert Spieler
 - Tauben (bewegen sich vertikal auf und ab): Bei Kollision verliert Spieler
 - Pylon bzw. Verkehrshüttchen (Objekt bewegt sich nicht)
 - Müllhaufen (Objekt bewegt sich nicht)
 - Bushaltestelle als Plattform (Objekt bewegt sich nicht)
 - Booster:
 - Sneaker (bewegt sich nicht)

- Durch Einsammeln eines Sneakers hat der Spieler die Möglichkeit sich schneller fortzubewegen. Der Sprint-boost hält 5 Sekunden an.
 - Bus als weggehendes Objekt
 - Hauptziel:
 - schnellstmöglich zum Ziel zu kommen, d.h. pünktlich zur Schule zu kommen
 - Steuerung-info-Button (Hinweisfenster für Steuerung)

Links	„←“	„A“
Rechts	„→“	„D“
Springen	„↑“	„W“
- Level-Szene
 - Hier hat der User die Möglichkeit ein Level auszuwählen. Es wird insgesamt 3 Level geben
 - Level werden erst freigeschaltet, sobald der Spieler ein Level beendet hat. Als beendet gilt ein Level, wenn der Spieler bis ans Ziel gekommen ist.
- Game-Szene
 - Spieler spawnet auf der linken Seite im Spielfenster. Spieler sieht zu Beginn wie der Bus wegfährt. Sobald der Bus aus dem Spielfenster verschwindet, startet der Spieler und kann sich mit WAD od. den Pfeiltasten fortbewegen.
 - Während des Laufes erreicht der Spieler sowohl Hindernisse als auch Booster s.o.
- Pause-Szene
 - 3 Buttons
 - Zurück zum Startbildschirm
 - Level neustarten
 - Level fortsetzen
- End-Szene
 - Spieler erhält Hinweis, dass er/sie gewonnen hat.
 - Spielfeld verdunkelt sich und Modalfenster erscheint

Soll-Anforderungen (zusätzliche Features):

- Musik + Sound während des Spiels, sollen ebenfalls über Buttons im Startbildschirm ausgeschaltet werden können
- Vor dem Spiel soll ein Charakter ausgewählt werden können

Aus dem Konzept wurde folgendes ursprünglich (aus TS-2) geplantes entfernt:

➔ Steuerung: Sliden

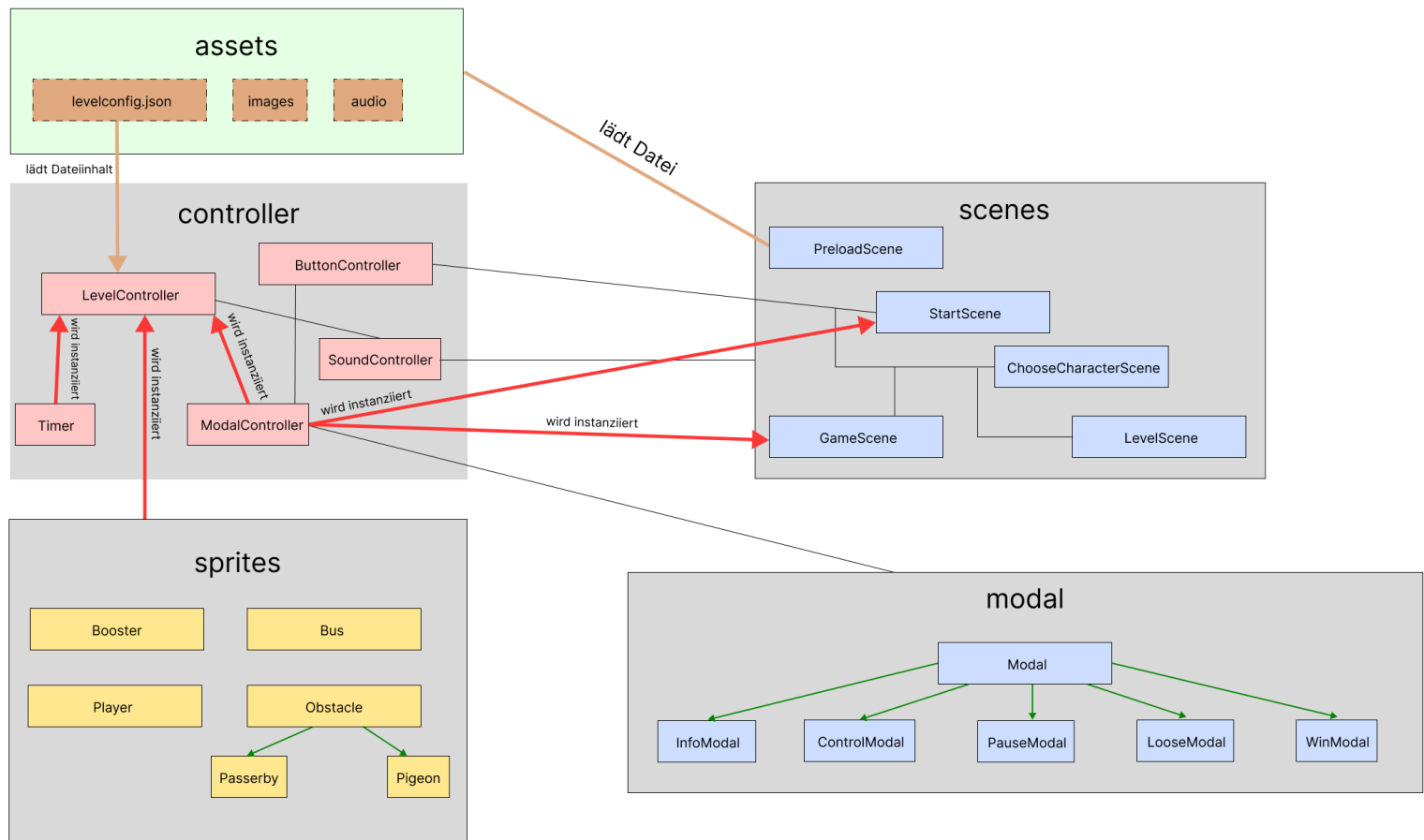
- Grund des Entfernens: Sinnhaftigkeit des Slides im Spiel wurde hinterfragt ebenso hätte das Bild des Player-Sprites verändert werden müssen, welches einen zeitintensiven Aufwand bedeutet hätte.

➔ Erstellen eines Highscoreboards anhand des Spielernamens sowie der Spielzeit. Hierbei sollte das Speichern über den Local Storage passieren.

- Grund des Entfernens: Es sind vermehrt Schwierigkeiten beim Einbauen eines input-Felds in Phaser aufgetreten. Da dies eine Soll-Anforderung war, fiel die Priorität auf das Refactoring des Codes als auf das Highscoreboard.

2 Softwarearchitektur

Die folgende Abbildung stellt die Projektstruktur des Spiels dar. Hierbei wurden die Dateien „index.html“, style.css, die „main.js“ und „phaser.js“ nicht berücksichtigt, da sie als Basis für das gesamte Projekt dienen. Der Schwerpunkt liegt auf den einzelnen JavaScript-Dateien, die die Spielmechanik steuern.



Übersicht der Projektstruktur

Assets: Im Assets Ordner finden sich alle Bilder, Sounds sowie die „levelconfig“-Datei, die Informationen zur Platzierung sowie Größe der Spielobjekte enthält.

Scenes: Die Scenes sind zuständig für das Darstellen der einzelnen Szenen, ebenso können sie auf Benutzerinteraktionen reagieren.

Controller: Die Controller stellen wichtige Methoden für die Verwaltung der Level, der Sounds, der Musik, der Modalfenster, des Timers sowie der Buttons zur Verfügung.

Sprites: Die Sprites bilden das Grundgerüst für die Spielobjekte Player, Bus, Booster, Passerby sowie Pigeon. Passerby und Pigeon erben von Obstacle. Die Objekte GarbageHeap, Busstop sowie TrafficCone, werden lediglich platziert, als unbeweglich eingestellt und mit dem Spieler sowie Plattformen kollidierbar gemacht, weshalb davon abgesehen wurde, Klassen von ihnen zu erstellen.

Modal: Die Modal-Klasse stellt eine Oberklasse und vererbt an die Klassen InfoModal, ControlModal, pauseModal, LooseModal sowie WinModal weiter.

3 Probleme und Lösung während der Entwicklung

Beim Erstellen des Spiels ist es vermehrt zu kleineren Problemen gekommen. Im Folgenden sind diese Probleme mit ihren jeweiligen Lösungen erläutert.

Modalfenster

Problem: Wenn ein Modalfenster geschlossen wird, sollen unterschiedliche Methoden ausgeführt werden. Beispielsweise soll bei der GameScene der Timer fortgeführt werden, sobald das Modalfenster geschlossen wurde.

Lösung: Wenn das Modalfenster geschlossen wird, wird ein Callback an die Schließen-Methode des Modals übergeben. In der Schließen-Methode wird dieser Callback dann ausgeführt. So ist es möglich beispielsweise die Methode resumeGame() in LevelController von der Modalklasse aus aufzurufen.

Problem: Das Modalfenster wird verschoben, wenn der Spieler sich von der Startposition wegbewegt.

Lösung: Um sicherzustellen, dass das Modalfenster seine Position beibehält und sich nicht mit der Kamera bewegt, wird der scrollFactor auf 0 gesetzt. Dadurch wird das Modalfenster von der Kamera entkoppelt und es bleibt statisch an seiner Position.

Obstacle (Hindernisse)

Problem: Bei den Obstacle hat das Anwenden von set.immovable() verursacht, dass die Obstacle nicht mehr mit den Plattformen kollidieren.

Lösung: Statt die Obstacle einer normalen Gruppe hinzuzufügen, werden sie nun einer statischen Gruppe hinzugefügt. Die Plattformen gehören ebenfalls einer statischen Gruppe an.

ModalController

Problem: Statusabfrage über eine globale Variable modalActive

Lösung: Es wurde eine ModalController-Klasse erstellt, die zuständig für die Verwaltung von Methoden für die Modalfenster ist. Darunter gehört, das Erstellen von Modalfenstern sowie das Setzen des Status, ob ein Modalfenster geöffnet ist. Das Modularisieren der Modalklassen trägt zur Wartbarkeit des Codes bei.

Sound + Musik

Problem: Sound und Musik sollen nicht in jeder Szene neu erstellt werden. Die Vermeidung von der Verwendung von globalen Variablen stellt hier das Ziel dar.

Lösung: Sound und Musik wird mithilfe des Singleton Patterns verwaltet. Das bedeutet, dass nur **ein** Objekt von SoundController erstellt und von Szene zu Szene weitergegeben wird.

4 Bilderquellen:

Dieses Spiel wurde im Rahmen des Moduls Rich Media Anwendungen erstellt und ist nicht für eine Veröffentlichung vorgesehen. Nach dem intensiven und erfolglosen Suchen von lizenzfreien Grafiken, die zu dem Aussehen meines Spiels passen würden, wurde der Entschluss getroffen, einige Grafiken trotz Lizenzpflicht zu nutzen. Diese sind im Folgenden mit URL gekennzeichnet.

Player1: <https://www.deviantart.com/boonzeet/art/Pokemon-Sword-Gloria-Female-Player-Gen-4-OW-V2-842639028>

Player2: <https://www.pinterest.de/pin/211174974559476/>

Bushes: <https://de.vecteezy.com/vektorkunst/22908198-pixel-straucher-oder-gebusch-mit-grun-und-laub>

Bussign: [Bushaltestellenschild mit pixel-art-stil | Premium-Vektor \(freepik.com\)](#)

Busstop: [SANTRIA! A quirky 2D-JRPG inspired by Earthbound and Twin Peaks \(tigsources.com\)](#)

Trees: https://www.pngitem.com/middle/hbxmJxT_transparent-pixel-tree-png-tree-png-download/

Ground: <https://www.nicepng.com/maxp/u2q8a9r5r5i1q8r5/>

School: https://de.freepik.com/vektoren-premium/schulgebaeudearchitektur-im-pixel-art-stil_49459402.htm

House: <https://stock.adobe.com/de/images/brick-house-pixel-art-brickwork-cottage-traditional-architecture-8-bit-sprite-game-development-mobile-app-isolated-vector-illustration/507971907>

TrafficCone: <https://stock.adobe.com/de/images/pixel-art-traffic-cone-vector-icon-for-8bit-game-on-white-background/488687564>

Passerby: <https://www.pinterest.de/pin/11118330323414038/>

Pigeon: <https://www.shutterstock.com/de/image-illustration/flying-bird-animation-pixel-art-646008130>

garbageHeap: https://www.freepik.com/premium-vector/pixel-art-isolated-junk-garbage_9338875.htm

Sneaker: <https://www.behance.net/gallery/87197241/Low-High-Force-Pixel-Art>

Clouds: https://www.freepik.com/free-vector/flat-design-pixel-art-cloud-illustration_38528396.htm#query=pixel%20art%20cloud&position=6&from_view=keyword&track=ais&uuid=7d00f565-5675-449b-bc2e-c4712e4b238c (free license)

Bus: selbst erstellt

Buttons selbst erstellt

Titles: selbst erstellt

Audio:

Startszene, Levelszene:

introMusic: <https://tallbeard.itch.io/music-loop-bundle> (freie Nutzung mit Verlinkung)

buttonClick: https://www.youtube.com/watch?v=YNSbL-Cek1c&ab_channel=BoostSound

Gameszene:

gameplayMusic: <https://tallbeard.itch.io/music-loop-bundle> (freie Nutzung mit Verlinkung)

winModal: <https://pixabay.com/sound-effects/you-win-sequence-1-183948/> (Free for use)

looseModal: <https://pixabay.com/sound-effects/negative-beeps-6008/> (Free for use)